COMPUTER ENGINEERING

QUEZON CITY

| | | | |
|---|---|---|---|
| **COURSE CODE/TITLE:** | CPE010 / Data Structures and Algorithms | **SECTION:** | CPE21S3 |
| **DATE PERFORMED:** | 09/10/2026 | **DATE SUBMITTED:** | 09/22/2026 |
| **NAME:** | Lance Cristopher S. Salcedo | **INSTRUCTOR:** | Engr. Jimlord Quejardo |

**HANDS ON ACTIVITY 8.1** **SORTING ALGORITHMS PT 2**

**1. PROCEDURE OUTPUT**

**Code + Output**

**Code:**

**main.cpp:**

```cpp
#include <iostream>
#include "sortingAlgo2.h"
#include <cstdlib>
#include <time.h>

const int maxSize = 100;

int main(){
    int dataArr[maxSize];
    srand(time(0));
    for (int i = 0; i < maxSize; i++ ){
        dataArr[i] = rand() % 10 + 1;
    }

    std::cout<<"Unsorted list:"<<std::endl;
    for (int i = 0; i < maxSize; i++ ){
       std::cout<< dataArr[i] <<" ";
    }

    std::cout<<"\nSorted List:"<<std::endl;
```

**sortingAlgo2.h:**

```
    for (int i = 0; i < maxSize; i++ ){
        std::cout<< dataArr[i] <<" ";
    }

    return 0;
}
```

**Output:**

```
Unsorted list:
5 4 1 9 2 9 3 7 5 7 4 10 7 2 8 8 4 7 3 5 2 2 2 3 5 4 9 1 9 7 5 5 1 1 9 3 2 10 5 3 1 3 6 3 6 6 8 1 10 6
 9 4 5 6 9 2 3 3 2 4 6 9 4 6 5 8 9 10 10 5 5 10 3 4 8 3 9 10 3 2 2 2 2 7 4 9 5 1 10 1 4 7 6 5 5 1 7 5
9 3
```

| | |
|---|---|
| **Observations** | **I noticed that it used srand to generate seeds for numbers to generate. I also noticed the for loop was used for the unsorted list.** |

**Table 8.1 Array of Values for Sort Algorithm Testing**

| | |
|---|---|
| **Code + Output** | **Code:**<br>**main.cpp:**<br>`std::cout << "Shell Sort: ";`<br>`shellSort(dataArr, maxSize);`<br>**sortingAlgo2.h:** |

```
template <typename T>
void shellSort(T arr[], int arrSize){
    for(int interval = arrSize/2; interval > 0; interval /= 2) {
        for(int i = interval; i < arrSize; i++){

            int temp = arr[i];
            int j;

            for(j = i; j >= interval && arr[j - interval] > temp; j -= interval){
                arr[j] = arr[j - interval];
            }

            arr[j] = temp; // temp

        }
    }
}
```

**Output:**

```
Unsorted list:
5 4 1 9 2 9 3 7 5 7 4 10 7 2 8 8 4 7 3 5 2 2 2 3 5 4 9 1 9 7 5 5 1 1 9 3 2 10 5 3 1 3 6 3 6 6 8 1 10 6
 9 4 5 6 9 2 3 3 2 4 6 9 4 6 5 8 9 10 10 5 5 10 3 4 8 3 9 10 3 2 2 2 2 7 4 9 5 1 10 1 4 7 6 5 5 1 7 5
9 3
Sorted List:
Shell Sort: 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 3 3 3 3 3 3 3 3 3 3 3 3 4 4 4 4 4 4 4 4 4 5
5 5 5 5 5 5 5 5 5 5 5 5 5 6 6 6 6 6 6 6 6 7 7 7 7 7 7 7 7 8 8 8 8 8 9 9 9 9 9 9 9 9 9 9 10 10 10
 10 10 10 10 10
```

| | |
|---|---|
| **Observations** | **For shell sort, I noticed that it is similar to insertion sort by comparing and swapping elements. It uses a "gap" based on what number the gap is while decreasing it by using the interval. The interval is decreased by halves until it is sorted.**<br><br>**Time Complexity: O (N log N)** |

**Table 8.2 Shell Sort**

| | |
|---|---|
| **Code + Output** | **Code:**<br>**main.cpp:**<br>`std::cout << "\nMerge Sort: ";`<br>`mergeSort(dataArr, 0, maxSize - 1);`<br>**sortingAlgo2.h:** |

```cpp
template <typename T>
void merge(T arr[], int left, int middle, int right){

    int n1 = middle - left + 1;
    int n2 = right - middle;

    T L[n1];
    T R[n2];

    for(int i = 0; i < n1; i++){
        L[i] = arr[left + i];
    }

    for(int j = 0; j < n2; j++){
        R[j] = arr[middle + 1 + j];
    }

    int i = 0;
    int j = 0;
    int k = left;

    while(i < n1 && j < n2){
        if(L[i] <= R[j]){
            arr[k] = L[i];
            i++;
        }
        else{
            arr[k] = R[j];
            j++;
        }
        k++;
```

```
        k++;
    }
    //L
    while (i < n1) {
        arr[k] = L[i];
        i++;
        k++;
    }
    //R
    while (j < n2) {
        arr[k] = R[j];
        j++;
        k++;
    }

}
```

```
template <typename T>
void mergeSort(T arr[], int left, int right){
    //If N=1, Arr is already sorted
    if( left >= right){
        return;
    }

    int middle = (left + right)/2;

    mergeSort(arr, left, middle);
    mergeSort(arr, middle + 1, right);
    merge(arr, left, middle, right);

}
```

**Output:**

```
Merge Sort: 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 3 3 3 3 3 3 3 3 3 3 3 3 3 4 4 4 4 4 4 4 4
4 5 5 5 5 5 5 5 5 5 5 6 6 6 6 6 6 7 7 7 7 7 7 7 8 8 8 8 8 8 9 9 9 9 9 9 9 9 9 9 9 10 10 10 10
10 10 10 10 10 10 10
```

| | |
|---|---|
| **Observations** | **For Merge Sort, I noticed that it used a sort to separate each group. It is using a "divide-and-conquer" paradigm to sort. It uses recursion and later merges them into 1.** |

| | |
|---|---|
| | Time Complexity: O (N log N) |

Table 8.3 Merge Sort

| | |
|---|---|
| Code + Output | Code:<br>main.cpp:<br>`std::cout << "\nQuick Sort: ";`<br>`quickSort(dataArr, 0, maxSize - 1 );`<br>sortingAlgo2.h:<br>`template <typename T>`<br>`int partition(T arr[], int low, int high){`<br>`//declare pivot at last index`<br>`int pivot = arr[high];`<br>`//lagging tracker`<br>`int i = low - 1;`<br>`//leading traversal`<br>`for(int j = low; j < high; j++){`<br>`if(arr[j] <= pivot){`<br>`i++;`<br>`std::swap(arr[i], arr[j]);`<br>`}`<br>`}`<br>`//moving pivot`<br>`std::swap(arr[i + 1], arr[high]);`<br>`return i + 1;`<br>`}` |

```
template <typename T>
void quickSort(T arr[], int low, int high){
    if (low < high){
        int pivot = partition(arr, low, high); //pivot

        quickSort(arr, low, pivot - 1);
        quickSort(arr, pivot + 1, high);
    }
}
```

**Output:**

```
Quick Sort: 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 3 3 3 3 3 3 3 3 4 4 4 4 4 4 4 4 4 4 4 4 5 5 5 5 5
5 5 5 5 6 6 6 6 6 6 6 6 6 6 7 7 7 7 7 7 7 7 8 8 8 8 8 8 8 8 8 9 9 9 9 9 9 9 9 9 9 9 10 10 10 10
10 10 10 10 10 10 10
```

| | |
|---|---|
| **Observations** | **For Quick Sort, it is similar to the merge sort where it uses a "divide-and-conquer" paradigm. I noticed that it uses a partition and pivot to compare whether it needs a swap or not.**<br><br>**Time Complexity: O (N log N)** |

**Table 8.4 Quick Sort**

## 2. ANSWERS TO SUPPLEMENTAL ACTIVITY

| | |
|---|---|
| **Problem** | **Problem 1**: Can we sort the left sub list and right sub list from the partition method in quick sort using other sorting algorithms? Demonstrate an example. |
| **Code** | **supplementary.cpp:** |

```cpp
#include <iostream>
#include "supplementaryAlgo2.h"

void displayArr(int arr[], int maxSize);

int main(){

    //Problem #1:
    int dataArr[] = {34, 7, 23, 32, 5, 62};
    int maxSize = sizeof(dataArr) / sizeof(dataArr[0]);

    std::cout << "Original Array: ";
    displayArr(dataArr, maxSize);

    int swap = partition(dataArr, 0, maxSize - 1);
    quickSort(dataArr, 0, swap - 1);
    std::cout << "Left Partition: " << std::endl;
    displayArr(dataArr, maxSize);

    quickSort(dataArr, swap + 1, maxSize -1);
    std::cout << "Right Partition: " << std::endl;
    displayArr(dataArr, maxSize);

    std::cout << "Sorted Array: ";
    for (int i = 0; i < maxSize; i++ ){
       std::cout<< dataArr[i] <<" ";
    }
    std::cout << "\n";
```

```cpp
    //Problem #2:

    return 0;
}
void displayArr(int arr[], int maxSize){
    for(int i = 0; i < maxSize; i++){
        std::cout << arr[i] << " ";
    }
    std::cout << std::endl;
}
```

| Output | |
|---|---|

Task 8.5 Supplementary Problem 1

| Problem | **Problem 2**: Suppose we have an array which consists of {4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19, 74}. What sorting algorithm will give you the fastest time performance? Why can merge sort and quick sort have O(N • log N) for their time complexity? |
|---|---|
| Code | **supplementary.cpp:**<br>`//Problem #2:`<br>`int dataArr[] = {4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19, 74};`<br>`int maxSize = sizeof(dataArr) / sizeof(dataArr[0]);`<br><br>`std::cout << "Original Array: ";`<br>`displayArr(dataArr, maxSize);`<br><br>`quickSort(dataArr, 0, maxSize - 1);`<br>`std::cout << "Quick Sort: ";`<br>`displayArr(dataArr, maxSize);`<br><br>`mergeSort(dataArr, 0, maxSize - 1);`<br>`std::cout << "Merge Sort: ";`<br>`displayArr(dataArr, maxSize);`<br><br>`return 0;`<br>**Questions:**<br>1. What sorting algorithm will give you the fastest time performance?<br>2. Why can merge sort and quick sort have O(N log N) for their complexity?<br>**Answers:**<br>1. The sorting that gives me the fastest performance is Quick Sort.<br>2. Both of them have the “divide-and-conquer” paradigm. |
| Output | `Original Array: 4 34 29 48 53 87 12 30 44 25 93 67 43 19 74`<br>`Quick Sort: 4 12 19 25 29 30 34 43 44 48 53 67 74 87 93`<br>`Merge Sort: 4 12 19 25 29 30 34 43 44 48 53 67 74 87 93` |

Task 8.6 Supplementary Problem 2

**Conclusion:**

I was able to finish the hands-on activity. The supplementary activity was not complicated and easy to follow and I did not have a hard time. I also learned how each sorting algorithm works with quick sort being my favorite out of the three.

## 3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

COMPUTER ENGINEERING

QUEZON CITY

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

**Tools Used:** **Gemini**

*Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.*

**Purpose of Use:** **Answering questions and observations**

*Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.*

**Extent of Use:** **I used Gemini for the given questions and observations except the code in the supplementary activity, all of the codes and sentences given are mine only.**

*Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.*

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.